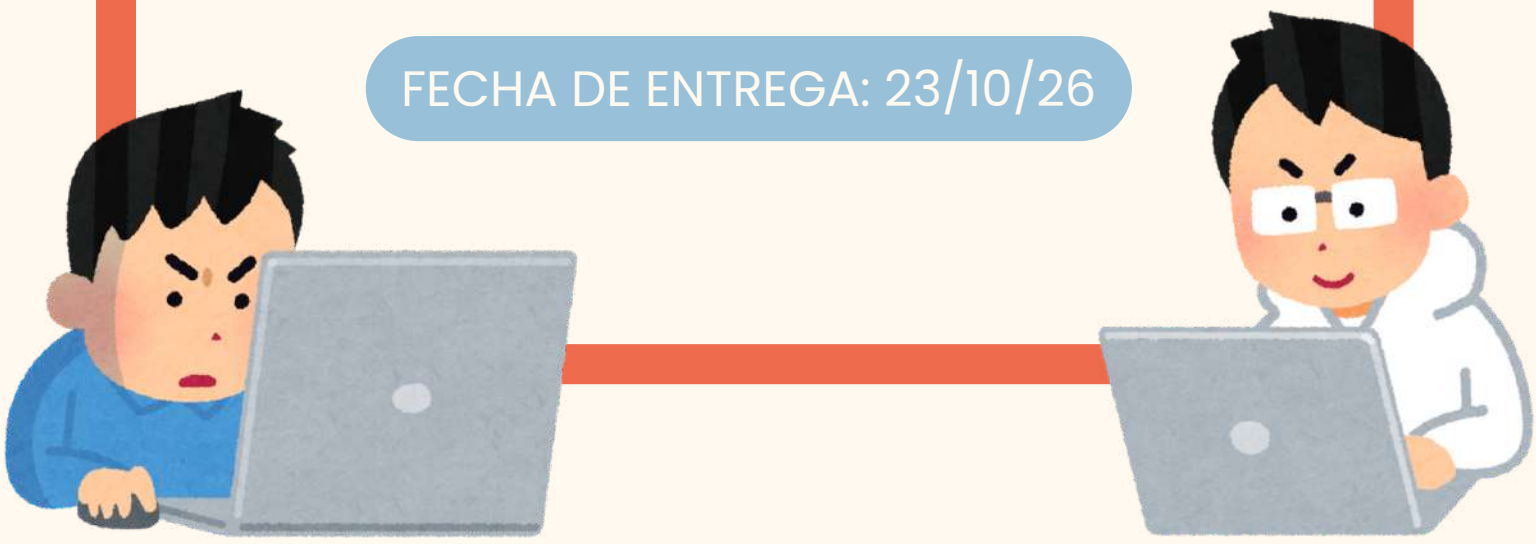
CRUZ HERNÁNDEZ LUIS ÁNGEL
NO. CUENTA: 121003056

DECISIONES Y REPETICIONES

FECHA DE ENTREGA: 23/10/26

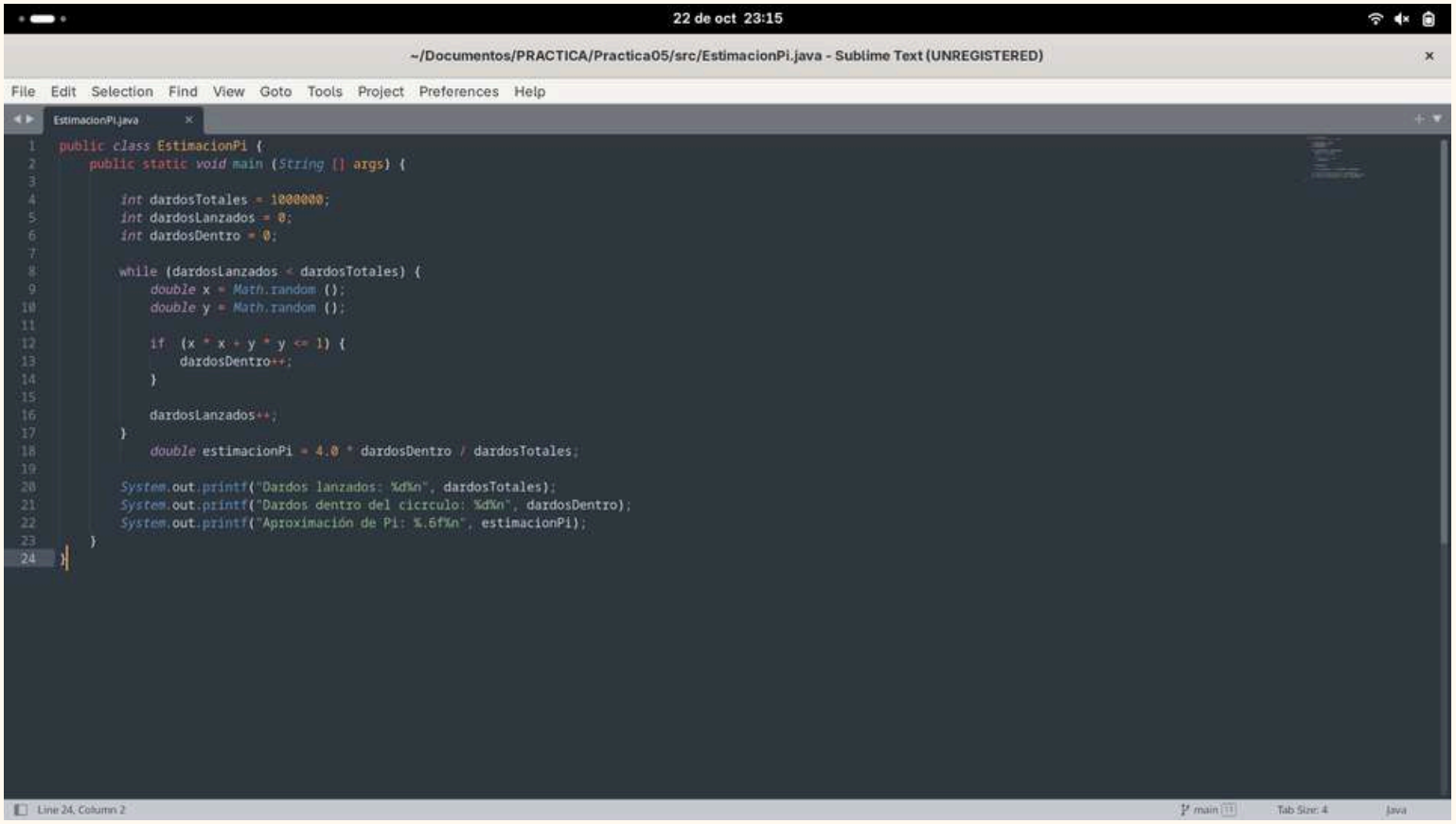


PARTE 1: CALCULANDO Π A DARDAZOS.

Buenas buenas un gusto volver a vernos en prácticas jeje, bueno empecemos con esto que no voy a dormir jaja. Iniciamos esta práctica sabiendo dos cosas fundamentales, primero el **"método Monte Carlo"** y se usa para estimar integrales, simular fenómenos físicos o calcular riesgos financieros. Para esto yo no sabia de este método, fue nuevo para mi así que tuve que analizar un poco más para poderlo entender bien, lo siguiente que hice fue que me metí al link que hay en el archivo de la práctica, ya que ahí me enseñan que es y como funciona, además que en el laboratorio lo vimos y entendí mejor esa parte de como funciona. Lo siguiente fue practicar con while ya que lo ocuparíamos en esta práctica y también el uso de if y else. Sabiendo esto ahora si empecemos con la primera actividad (spoiler costo mucho trabajo).

ACTIVIDAD 1.1 – UN NÚMERO FIJO DE DARDOS

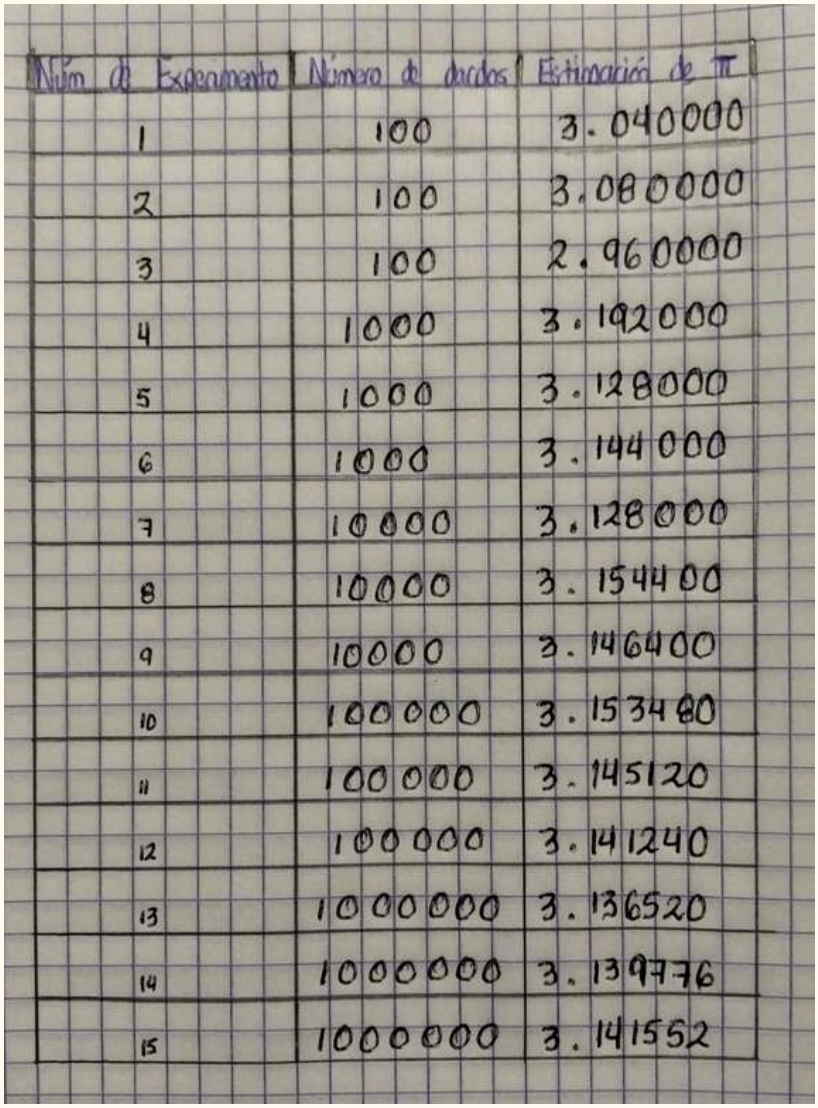
Para empezar esta actividad, teníamos que conocer el **"método Monte Carlo"** y cómo funciona para poderlo aplicar en el código, y ahora si empezar a programar y acabar con el estrés, para esto empece a ver el funcionamiento en la página proporcionada y además preguntándole a un compañero para iniciar el código. Para esto quiero recalcar que hice uso de IA pero no para resolver y dejar que ella hiciera la práctica, la ocupé ya que una de las partes de la práctica no entendí bien a que se refería para hacer el código, la utilice más como ayuda y entender mejores las cosas, con esto hablado y visto bien los puntos a empezar a programar. Iniciamos creando un **".java"** abrimos nuestro Sublime text y empezamos a escribir, primero antes que nada la estructura al iniciar un nuevo código y su nombre en este caso fue **"EstimacionPi"**, luego declaramos las variables que vamos a ocupar, en este caso, fueron tres **"int"** los cuales me ayudarían para realizar el **"método Monte Carlo"**, declaramos sus datos correspondientes, en este caso lo veríamos como dardos y que los estamos lanzando, así que cada una de las variables llevarían los datos correspondientes en este casos fueron **"dardosTotales"**, **"dardosLanzados"** y **"dardosDentro"** los cuales ocuparíamos más adelante, después agregué un ciclo while para repetir el lanzamiento de los dardos, el ciclo se ejecuta mientras el número de dardos lanzados sea menor que el número total de dardos que declaré, en cada vuelta que se da se lanza un dardo y se suma 1 al contador de dardos lanzados, cuando el contador llega al total, la condición deja de cumplirse y el ciclo se detiene, por lo que ya no se lanzarían más dardos. Con esto cumplo la condición de agregar un while al código para saber el ciclo y como se detiene ese ciclo, a continuación agrego el código, este nos sera útil para poder explicar lo siguiente, porque además de agregar el while no solo se quedaría con lo explicado antes si no que tenemos que agregar otros datos y ver a detalle cada uno para ver bien el funcionamiento del código.



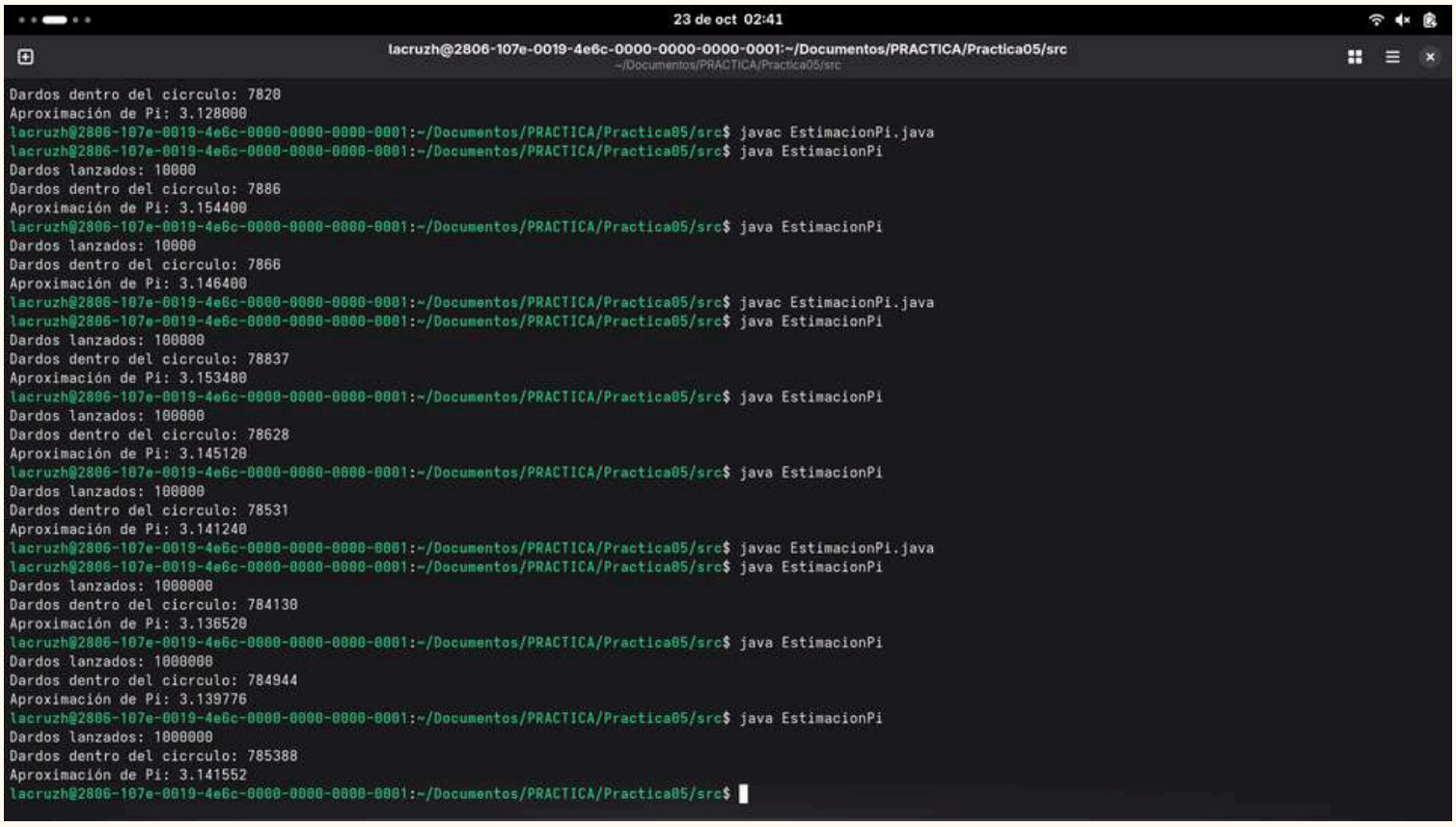
Podemos observar ya con el código escrito varias cosas que no explique, primero antes que nada se tenia que utilizar un ciclo while el cual ejecuta un bloque de código mientras se cumpla una condición. Antes del ciclo declaré tres variables: dardosTotales, que guarda cuántos dardos quiero lanzar; dardosLanzados, un contador que inicia en 0; y dardosDentro, que cuenta los dardos que caen dentro del cuarto de círculo, que son las variables previamente vistas pero sabiendo que hace cada una. Continuando, la condición del ciclo es dardosLanzados < dardosTotales, es decir, el ciclo se repite mientras falten dardos por lanzar, en cada vuelta se lanza un dardo: se generan sus coordenadas x y y con Math.random(), que representan un punto al azar dentro del cuadrado de 1 × 1, y agregamos un if que revisa si x * x + y * y <= 1; si se cumple, el dardo cayó dentro del cuarto de círculo y aumenta dardosDentro, al final de cada vuelta se suma 1 a los dardosLanzados, este dato es indispensable, porque cuando el contador llega a dardosTotales la condición deja de cumplirse y el ciclo se detiene; si ese dato no se actualizara, el ciclo sería infinito. Con esto se comprueba el uso del while y las variables declaradas, ademas que igualmente explicando esto podemos darnos cuenta el porque de cada cosa, más que nada mejor explicado para que se entienda bien (o bueno eso creo que se entiende bien la explicación del while). Bueno hasta ahorita todo bien ya se declaro y como funciona el ciclo de los dardos, pero ahora se calcularia la aproximación de PI, que junto con esto imprimiria cuantos dardos cayeron dentro del circulo, el total de dardos que lanzariamos, y por ultimo la aproximación de PI con 6 digítos, bueno visto eso, y sabiendo cada cosa seguiriamos con la explicación del como fue para sacar la aproximación de PI.

Podemos observar que para sacar la aproximación o estimación de PI se saco por una fórmula, pero como llegamos a esa fórmula, pues la verdad no entendí muy bien al principio pero después de investigar, la explicación sería porque se calcula por el método Monte Carlo, el cual como vimos anteriormente es un cuadrado de 1 x 1 con un cuarto de círculo de radio 1, además vasandome en la fórmula proporcionada en el documento de la práctica se despejaría y nos quedaría π ≈ 4 × (dardos dentro / dardos totales), con esto solo se interpretaría para el código, que seria de la forma en la que la escribí 4.0 * dardosDentro / dardosTotales, se usa el 4.0 para que la división se haga con decimales, pues si la cantidad es entera Java descarta la parte decimal y ya no estaríamos viendo los demás datos de PI, ya como último imprimí el total de dardos, cuántos cayeron dentro del círculo y la aproximación de π con 6 decimales como es que lo pide la actividad. Con esto puedo explicar como fue para sacar la aproximación de PI.

Ya para terminar esta actividad fue aplicar mi código y generar una tabla donde se viera la cantidad de dardos lanzados, y hacerlo tres veces con diferentes cantidades de dardos, a continuación mi tabla:



Núm. de Experimento	Número de dardos	Estimación de π
1	100	3.040000
2	100	3.080000
3	100	2.960000
4	1000	3.192000
5	1000	3.128000
6	1000	3.144000
7	10000	3.128000
8	10000	3.154400
9	10000	3.146400
10	100000	3.153480
11	100000	3.145120
12	100000	3.141240
13	1000000	3.136520
14	1000000	3.139776
15	1000000	3.141552



Agrego también una captura para que se vea como me iba arrogando mi código los diferentes tipos de datos.

Para ahora si de si, terminar esta primera actividad contestare las siguientes dos preguntas:

¿Por qué tres ejecuciones con el mismo número de dardos dan resultados distintos?

Mi teoría que es magia, no es cierto jeje, pues más que nada es por el Math.random (), ya que este nos genera un número decimal al azar cada vez que se le llama, por cual por eso en cada ejecución las coordenadas de los dardos son diferentes, y cambia la cantidad de dardos que caen dentro del cuarto de círculo, como la estimación de π depende de esa cantidad, el resultado cambia en cada ejecución aunque el número total de dardos sea el mismo.

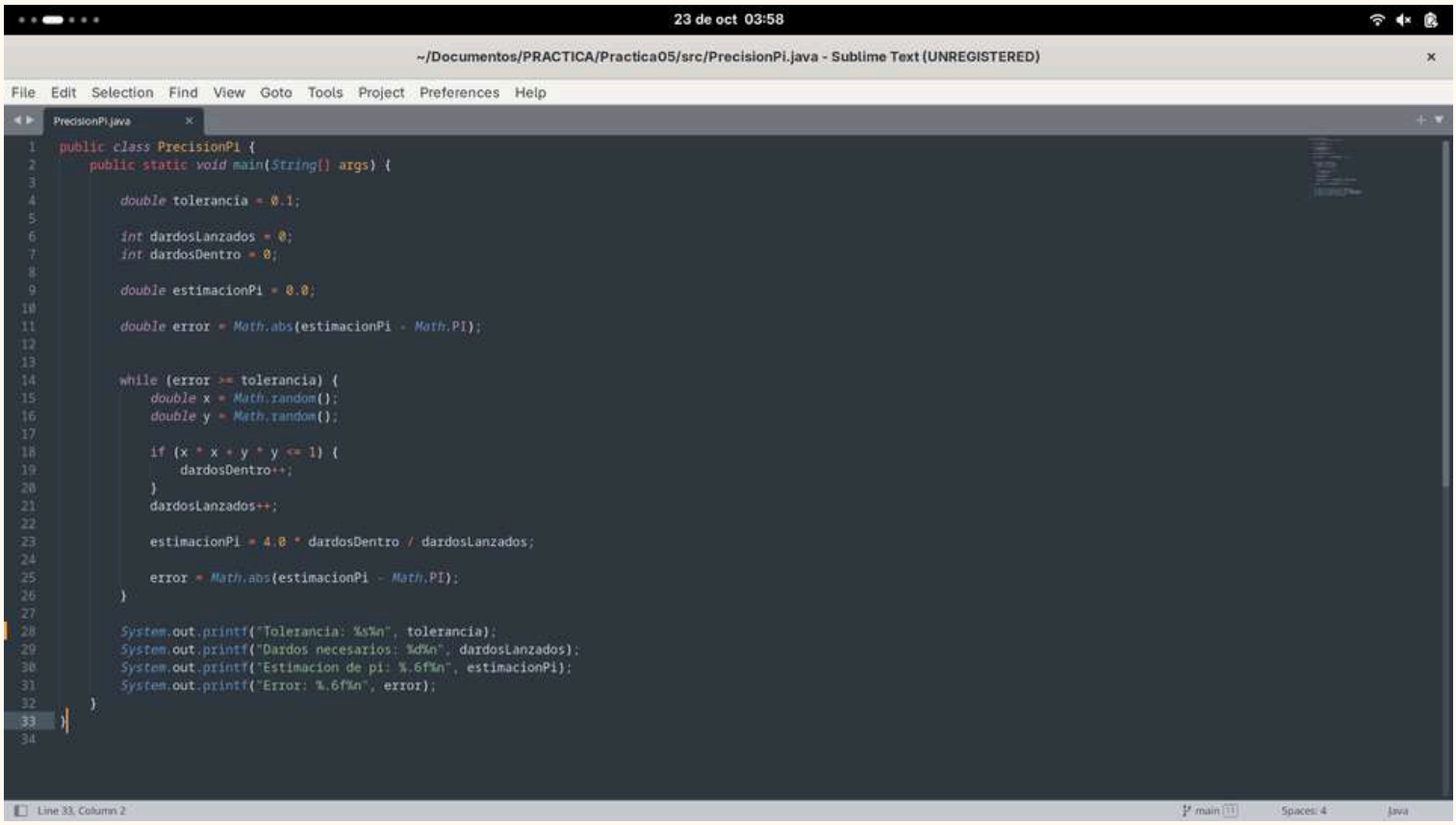
¿Qué pasa con la estimación conforme aumenta el número de dardos?

Lo podemos ver que cuando vaya aumentando de dardos se aproxima más al valor de π, mientras que con pocos dardos, la aleatoriedad influye mucho y los resultados cambian bastante de una ejecución a otra. Por ejemplo: con 100 dardos obtuve estimaciones de 3.04, 3.08 y 2.96, mientras que con 1,000,000 de dardos todas quedaron cerca de 3.14. Comprobando lo que dije al principio.

Con esto acabamos con la primera actividad, y es la primera, bueno a seguir con las siguientes.

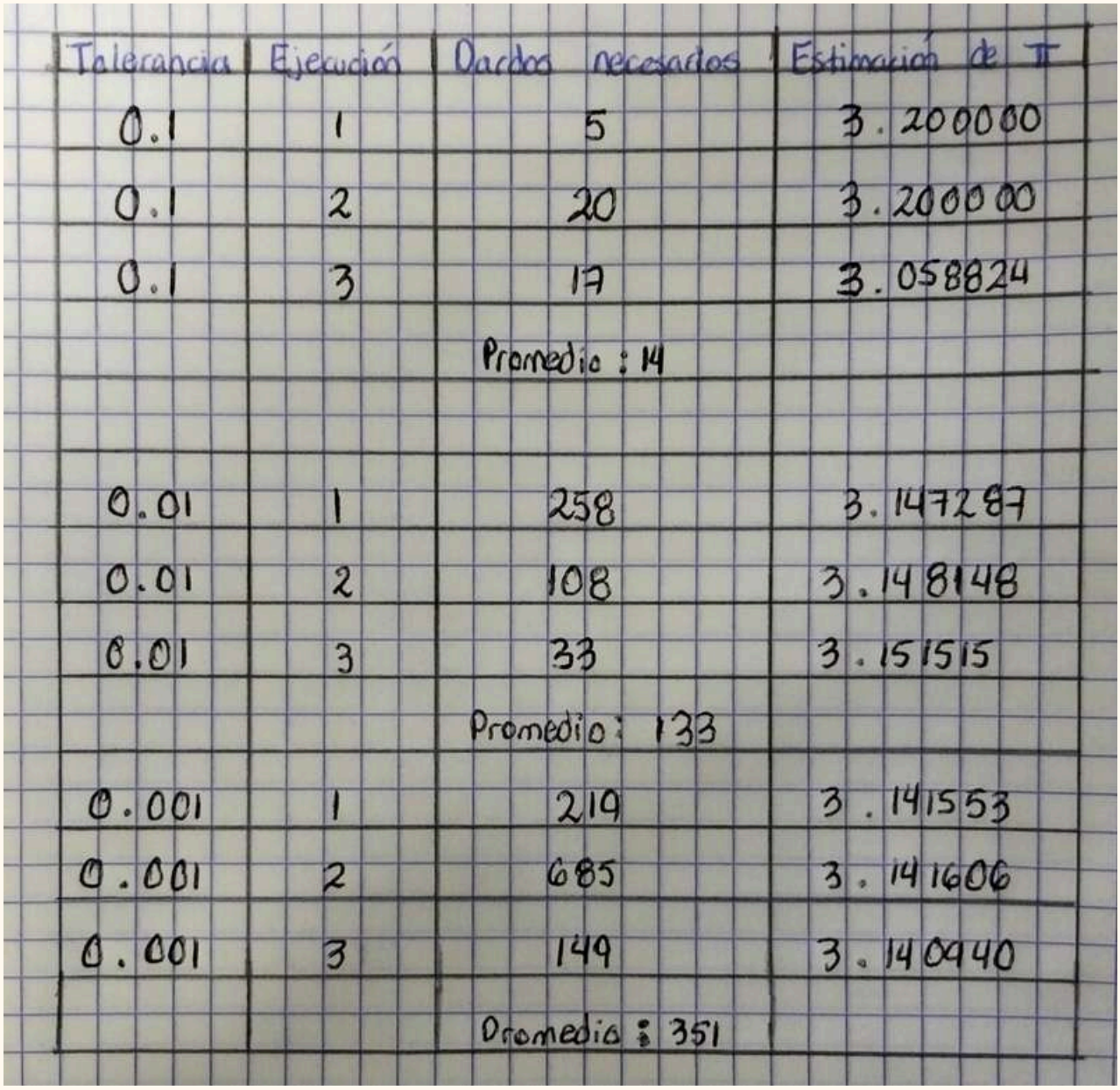
ACTIVIDAD 1.2 – REPETIR HASTA LLEGAR A LA PRECISIÓN DESEADA

Bueno continuamos que esto apenas comenzó y tenemos que entregar las cosas, continuamos con la siguiente parte que en este caso, sería con la siguiente actividad en la cual vamos a ver la precisión de PI, bueno para empezar como en la anterior actividad, creamos un archivo **“.java”** pero en este caso vamos a colocarle el nombre de **“EstimacionPi”** después de lo que ya sabemos de agregar la estructura y de colocar los datos correspondientes podemos iniciar con lo mero bueno que sería el código, en este caso, seria indicar nuestras variables y la primordial seria la tolerancia ya que esta será el error máximo que vamos a aceptar en este caso yo ocupe 0.1, seguimos agregando las otras variables que en este caso sería los mismos datos que ocupamos en la anterior actividad, que serian los **“dardosLanzados”** y **“dardosDentro”**, con esto en cuenta solo agregaríamos como último la estimación de PI y ya tendríamos listas cada una de las variables para empezar a desarrollar el código.

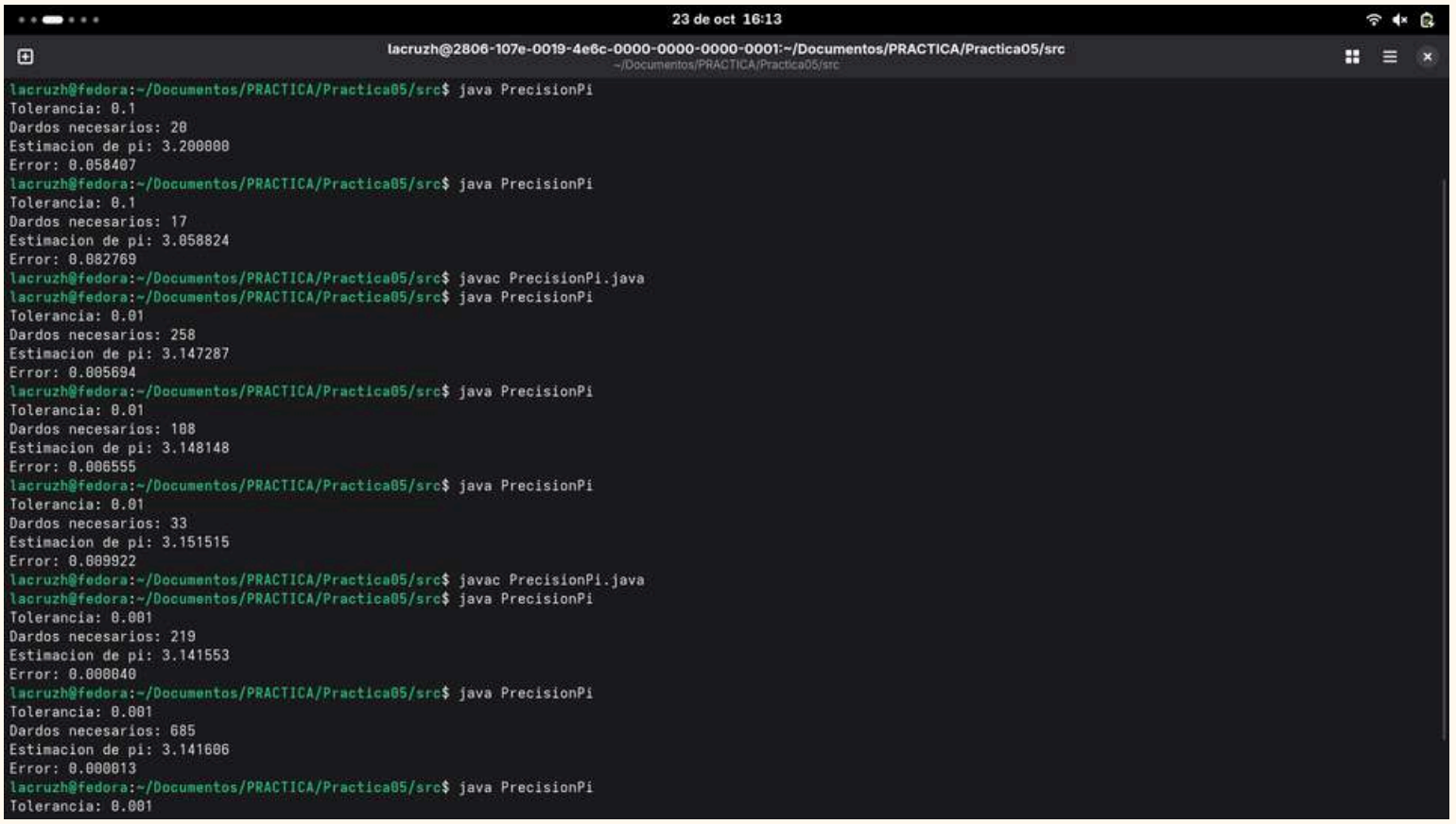


Observamos en esta captura de pantalla como esta digitado cada uno de los valores, y ahí va la explicación de cada uno y como es que llegue a cada uno de ellos, porque pues se ve bonito y todo y compila y toda la cosa, pero ok y de donde o como llego cada una de las cosas ahi, asi que a continuación, estaré explicando cada una de las cosas, para comprobar y que se vea que realice bien la actividad o bueno realice bien el código.

Primero declaramos la variable error, que guarda la distancia entre nuestra estimación de π y el valor real de π, guardado en la constante Math.PI. Para calcularla usamos Math.abs, que devuelve el valor absoluto y evita que el error sea negativo. Como estimacionPi inicia en 0.0, el error inicial es de aproximadamente 3.14, lo que permite que el programa entre al ciclo. Después agregamos el while, que usa la misma lógica del ejercicio anterior, pero ahora sigue lanzando dardos mientras el error sea mayor o igual que la tolerancia. Dentro del ciclo generamos las coordenadas x y y con Math.random(), y con el if revisamos si el punto cae dentro del círculo para aumentar dardosDentro. Luego aumentamos dardosLanzados, calculamos la nueva estimación de π con 4.0 * dardosDentro / dardosLanzados y actualizamos error con la misma fórmula del inicio, ahora usando la nueva estimación. Así, en cada vuelta el while revisa si ya se alcanzó la precisión pedida. Al terminar el ciclo, imprimimos con printf los 4 datos (tolerancia, dardos necesarios, estimación de π y error), que son los que usaremos para llenar la tabla, donde la tabla fue la siguiente cuando ya se corre el código:



Tolerancia	Ejecución	Dardos necesarios	Estimación de π
0.1	1	5	3.200000
0.1	2	20	3.200000
0.1	3	17	3.058824
		Promedio : 14	
0.01	1	258	3.147287
0.01	2	108	3.148148
0.01	3	33	3.151515
		Promedio: 133	
0.001	1	219	3.141553
0.001	2	685	3.141606
0.001	3	149	3.140940
		Promedio : 351	



Observamos dos cosas primero que la tabla que esta arriba, concuerdan con los resultados que nos arrogo el código, además que igualmente en la tabla se ve reflejado el promedio de los dardos de cada una de las tolerancias. Siguiendo y para poder terminar la segunda actividad juju, contestaremos lo siguiente:

¿Qué tanto variaron los dardos necesarios entre las tres ejecuciones de una misma tolerancia? ¿Te parece que el promedio representa bien lo que pasó?

Los dardos necesarios variaron bastante entre las tres ejecuciones de una misma tolerancia. Con tolerancia 0.1 se necesitaron entre 5 y 20 dardos; con 0.01, entre 33 y 258; y con 0.001, entre 149 y 685. En todos los casos el valor más grande fue de 4 a 8 veces el más pequeño. Esto sucede porque cada ejecución usa números aleatorios distintos, y el ciclo se detiene en cuanto la estimación queda por primera vez dentro de la tolerancia, algo que puede pasar muy pronto o mucho después según el azar.

El promedio no representa bien lo que pasó, sobre todo con 0.01 y 0.001. Con 0.01 el promedio fue 133, pero las ejecuciones fueron 33, 108 y 258; ninguna se parece realmente a 133. Con 0.001 el promedio fue 351, pero los resultados fueron 149, 219 y 685; el valor más grande (685) jala el promedio hacia arriba y ninguna ejecución se acerca a 351. Con 0.1 el promedio (14) se parece más a 17 y 20, pero tampoco a 5. Por eso, el promedio da una idea general (a menor tolerancia se necesitan más dardos: 14, 133, 351), pero oculta la gran variación entre ejecuciones.

Compara esta tabla con la de la actividad 1.1 ¿cuántos dardos necesitaste para llegar a un error menor a 0.001? ¿Concuerda con lo que observaste con un número fijo de dardos?. Si no concuerda, ¿por qué crees que pasa?

No concuerda. En la Actividad 1.2, con tolerancia 0.001, el programa necesitó solo entre 149 y 685 dardos (promedio 351) para lograr un error menor a 0.001. Sin embargo, en la Actividad 1.1 con 1,000 y 10,000 dardos ninguna de mis ejecuciones logró ese error (los errores fueron de 0.0024 como mínimo), y ni siquiera con 1,000,000 de dardos lo logré siempre: solo una de las tres ejecuciones (3.141552) tuvo error menor a 0.001, mientras que las otras dos tuvieron errores de 0.0051 y 0.0018. En total, solo 2 de los 15 experimentos de la Actividad 1.1 lograron esa precisión.

Creo que esto pasa por el momento exacto en que se detiene el ciclo. En la Actividad 1.2 el programa revisa el error después de cada dardo y se detiene la primera vez que el error baja de la tolerancia, aunque sea por casualidad. La estimación no se acerca a π de forma suave, sino que oscila alrededor de él, así que en algún momento puede pasar muy cerca de π con pocos dardos, y el ciclo se detiene justo ahí. Ese resultado es “afortunado” y no sería estable: si se siguieran lanzando dardos, el error podría volver a ser mayor. En cambio, en la Actividad 1.1 el programa lanza todos los dardos y solo se mira el resultado final, sin aprovechar ningún momento de suerte en el camino. Por eso, para lograr la misma precisión de forma confiable se necesitan muchos más dardos que los que indica la Actividad 1.2.

Con esto contestado podemos darle el cierre a la primera sección y podemos continuar con lo siguiente que serian la sección 2 con el cotizador de señor Pines, pero ya lo veremos en la siguiente diapositiva, mientras tanto fue todo para la primera parte, tarde mucho en descifrar muchas cosas, porque la verdad tengo muchas dudas acerca de cada cosa, pero mejor entregar aunque sea con dudas y ver después como puedo practicar para no perderme mucho, bueno esto debería estar escribiéndolo en la conclusión así que por ahora se queda así este pequeño comentario jeje.



PARTE 2: EL REGRESO DEL SEÑOR PINES

Para continuar con la práctica, volvemos a reencontrarnos con el señor Pines, pero ahora nos pide realizar cambios en nuestro cotizador que realizamos en la práctica pasada, con algunos cambios muy específicos, asi que manos a la obra que tenemos que ayudar el señor Pines si no perderá su dinero y no queremos eso.

Para iniciar en este nuevo cotizador, tendremos que retomar nuestro código pasado, en este caso ocupe el que nos proporciono nuestro laboratorista, ya que la verdad esta mejor estructurado y pues no perdernos mucho en el código y podamos atacar bien los puntos que se nos pide, al igual que hay que checar varias cosas. Bueno ahora a los que nos gusta programar.

ACTIVIDAD 2.1 – VERIFICAR LOS DATOS Y EL TIPO DE CLIENTE

Para iniciar visualice que mi programa tenga los datos correspondientes. Para eso usé una variable booleana llamada datosValidos, que empieza en true. Después, con cuatro condiciones if independientes, reviso cada dato, y si alguno falla, imprimo un mensaje de error y cambio datosValidos a false.
Los datos que elegí revisar y cómo los reviso:

- Capital (capital <= 0): un préstamo no puede ser de cero ni de un monto negativo, porque no tendría sentido financiero.
- Tasa anual (tasaAnual < 0): una tasa negativa significaría que el cliente paga menos de lo que pidió. Acepté el cero porque una tasa de 0% sí puede existir (un préstamo sin interés).
- Plazo en meses (plazoMeses <= 0): el plazo debe ser al menos de un mes. Además, si fuera cero, la mensualidad dividiría entre cero.
- Tipo de cliente (tipoCliente != 'P' && tipoCliente != 'E' && tipoCliente != 'R'): solo se permiten esos tres valores. Usé && porque el dato es inválido únicamente cuando no es ninguna de las tres letras permitidas.

Usé if separados (no else if) a propósito: así, si hay varios datos incorrectos al mismo tiempo, el programa imprime todos los errores y no solo el primero. Al final, todo el cálculo está dentro de if (datosValidos), de modo que solo se ejecuta cuando los cuatro datos son correctos.

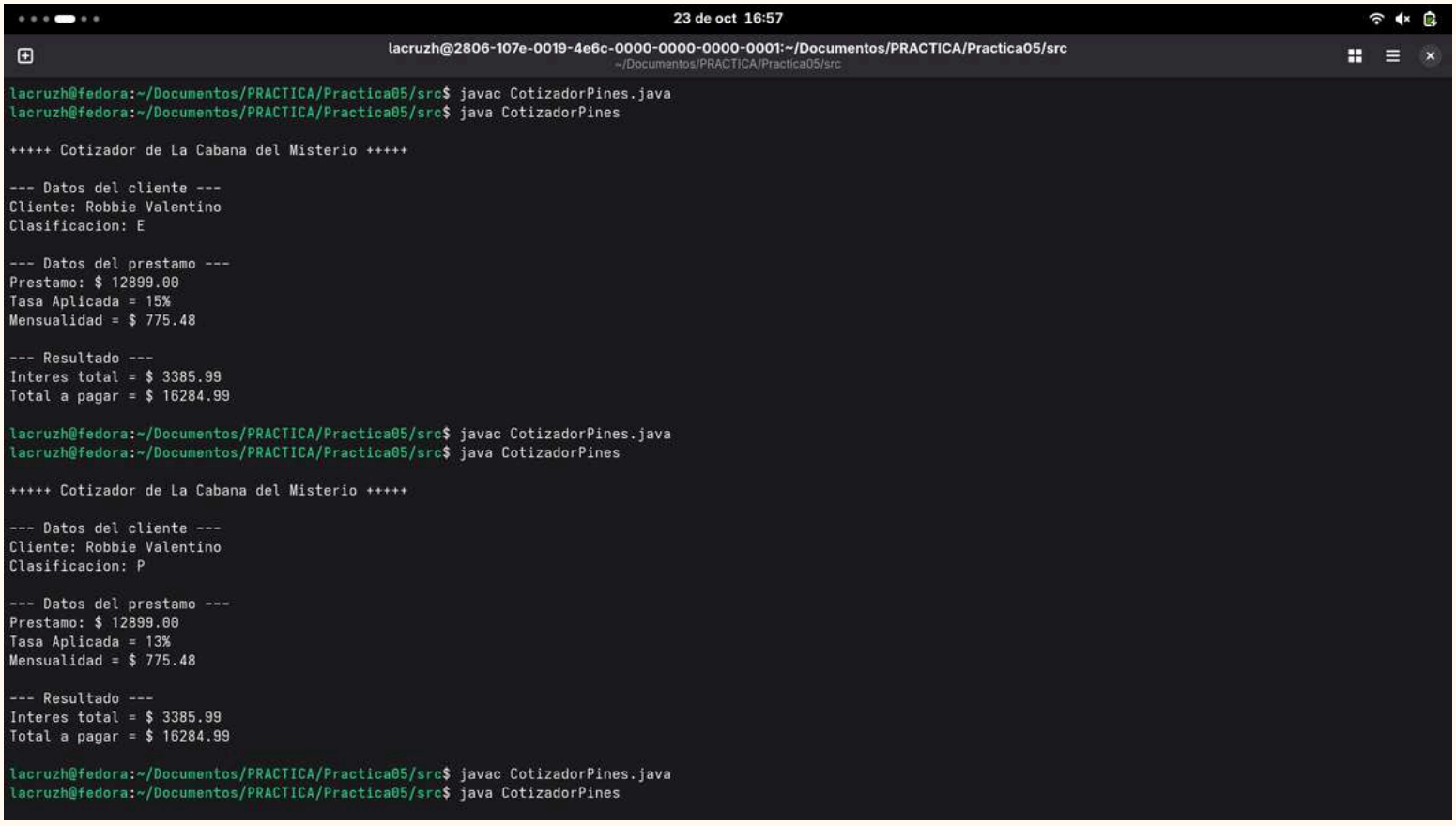
Mi estrategia fue partir de la tasa anual base (tasaAnual) y guardar el resultado en otra variable, tasaAplicada, para no perder el valor original. Usé una estructura if, else if y else:

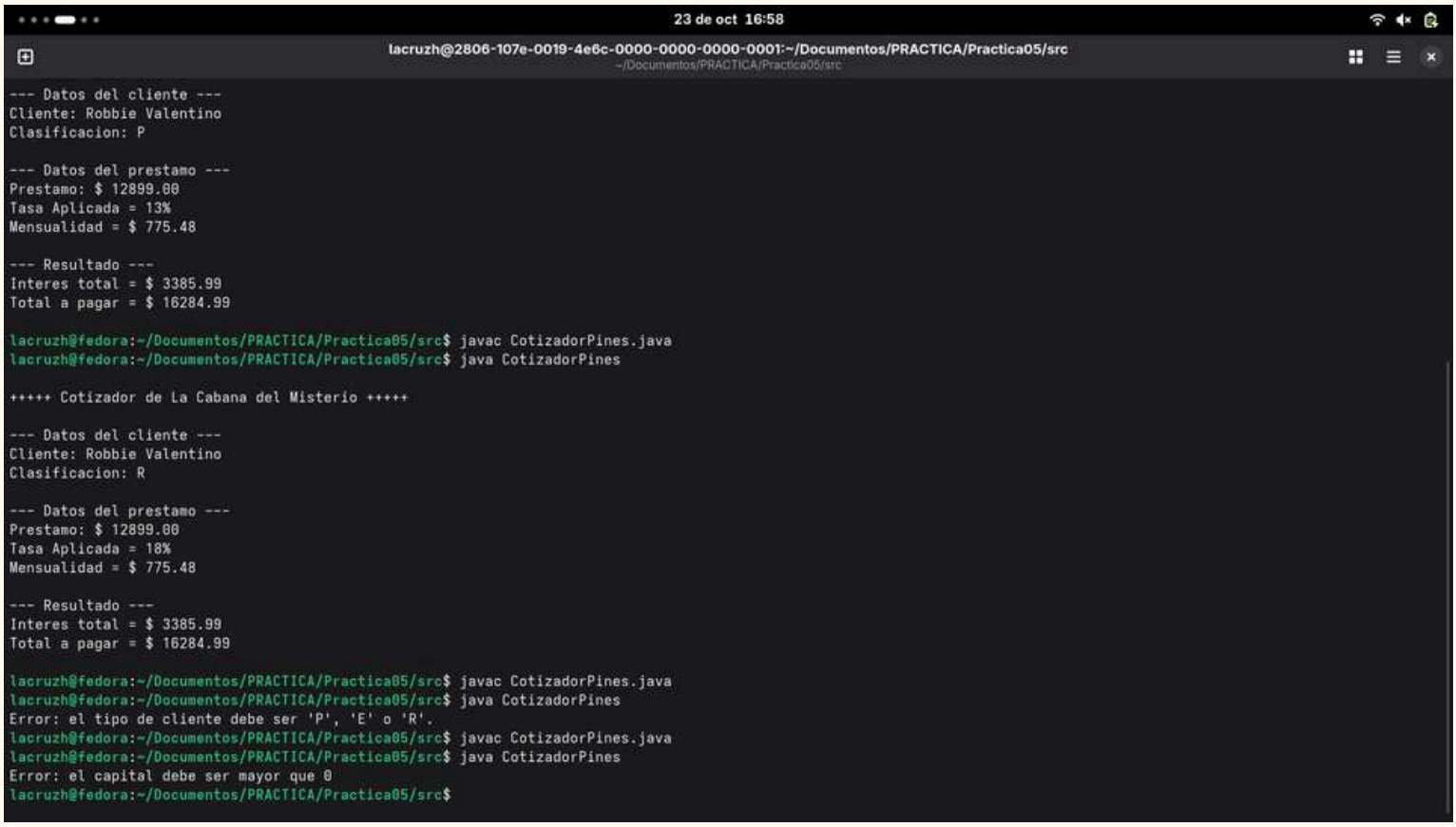
- Si el cliente es 'P' (Preferente), la tasa baja 2 puntos: tasaAnual - 2.
- Si es 'E' (Estándar), no cambia: tasaAnual.
- En cualquier otro caso ('R', Riesgoso), sube 3 puntos: tasaAnual + 3.

Pude usar else para el último caso, sin preguntar por 'R', porque la validación previa ya garantiza que el tipo solo puede ser P, E o R.
Con la tasa aplicada calculé el interés simple en tres pasos:

1. Convertí los meses a años: plazoAnios = plazoMeses / 12.0. Usé 12.0 y no 12 para que Java haga una división con decimales y no una división entera (21/12 daría 1, no 1.75).
2. Calculé el interés: interes = capital * (tasaAplicada / 100) * plazoAnios.
3. Calculé el total: totalApagar = capital + interes, y la mensualidad: mensualidad = totalApagar / plazoMeses.

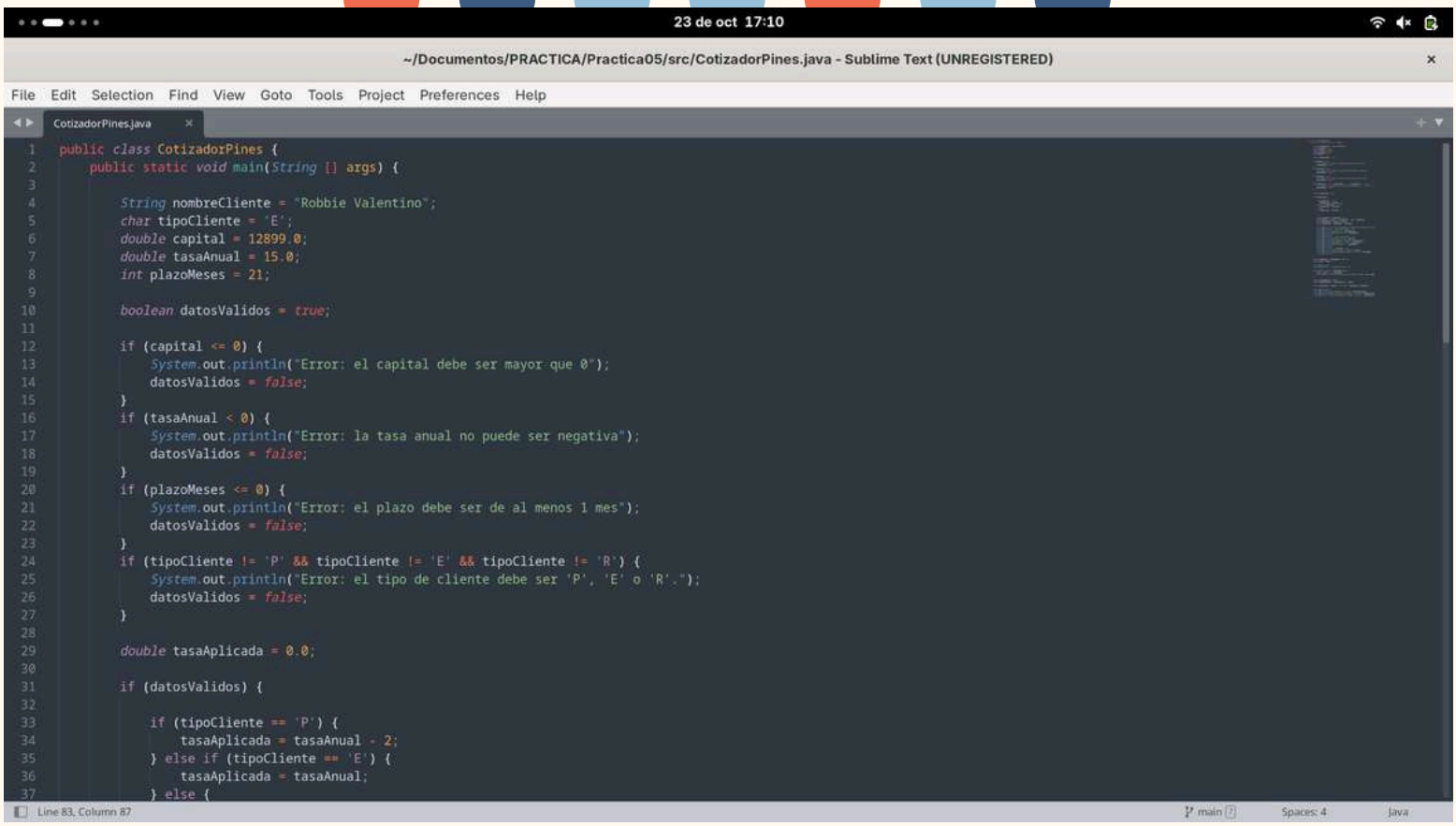
Ya por último, hice las ejecuciones correspondientes que me pide la práctica para que se compruebe que mi código realiza las cosas bien y no hay ningún fallo, para esto adjunto lo que me entrega mi terminal al aplicar mi código:

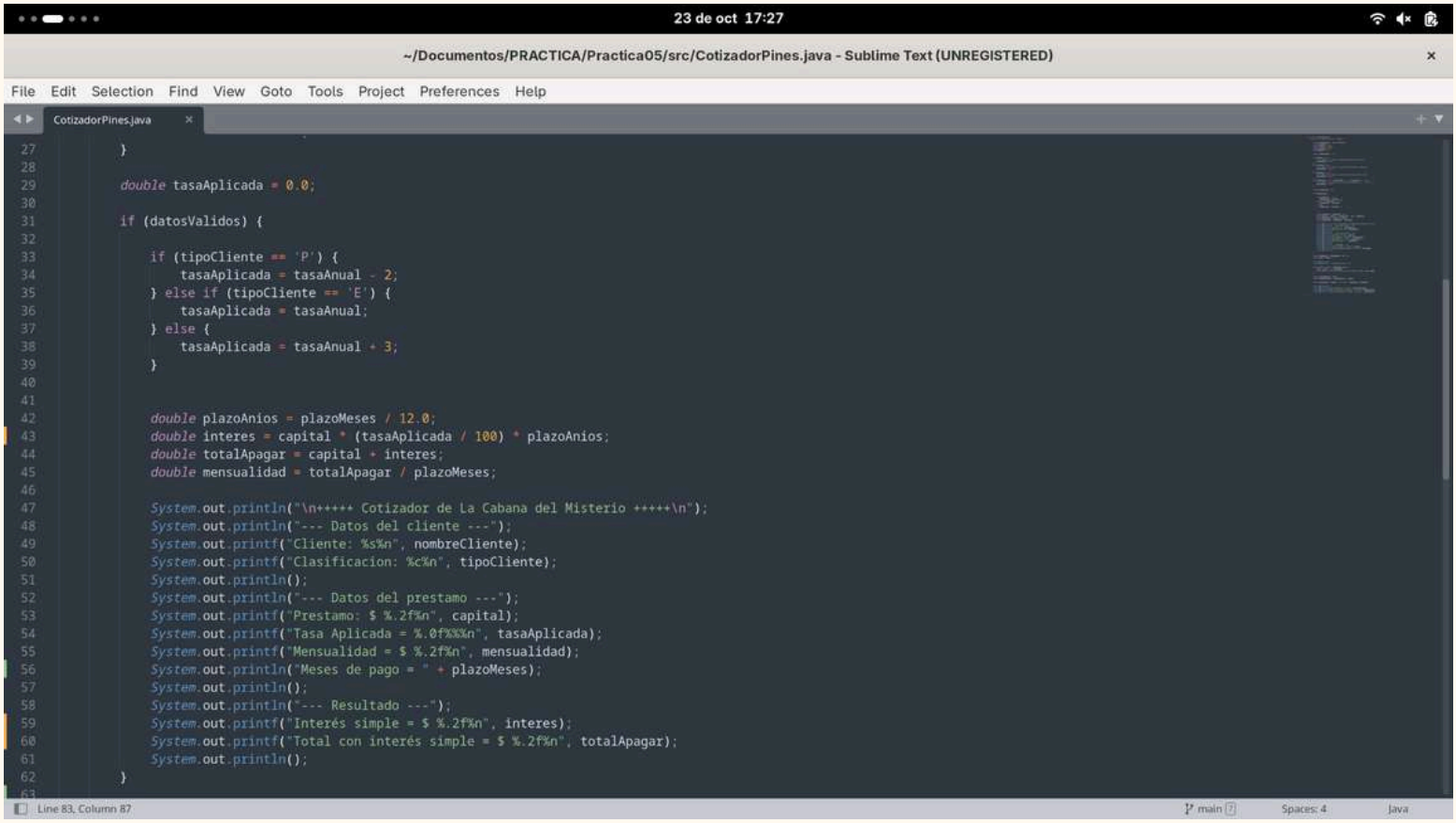




En las ejecuciones 1 a 3 se observa que, entre más riesgoso el cliente, mayor es la tasa aplicada, y por lo tanto mayores son el interés total, el total a pagar y la mensualidad. En las ejecuciones 4 y 5 el programa detectó el dato inválido, imprimió el mensaje correspondiente y no realizó ningún cálculo.

Podemos concluir la primera actividad donde tenia que colocar los diferentes datos extras, para que funcionara bien el código y no tuviera ningún problema en detectar cada uno de los errores, creo que la estaré regando un poco, pero en esta parte no agregue mi código como tal para que se checara que hice y porque salio el cada una de las cosas, así que a continuación mi código que si no como se comprueba, bajo de décimas y nonono, queremos ese 10 en la práctica jeje





Ahora si mi código cien por cien verídico y para que no se note que solo pongo lo que deja la terminal con esto ahora si que si, comprobamos como se realizo la actividad y como es que llegue a cada uno de los resultados correspondientes a esta primera actividad, así que ahora si para ya dejarme de cosas y que no tengas que leerte otra de mis biblias continuamos con la siguiente y ultima actividad.

ACTIVIDAD 2.2 – INTERÉS COMPUESTO

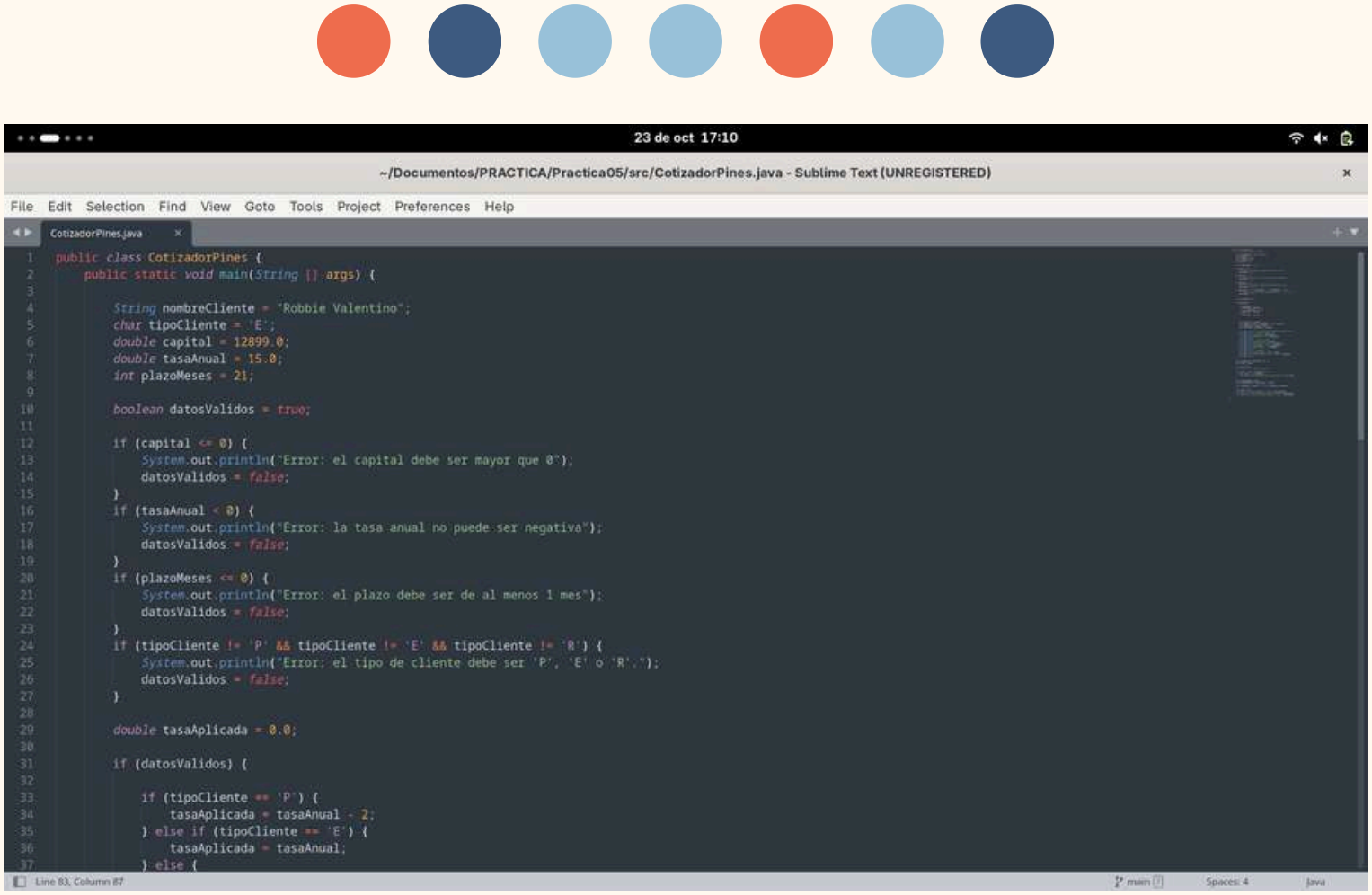
Para iniciar en esta sección, tuve que ver el dato de interés compuesto, ya que el interés de cada mes se suma a la deuda, y el mes siguiente se calcula sobre ese nuevo saldo. Para modelar seguí los pasos de la actividad:

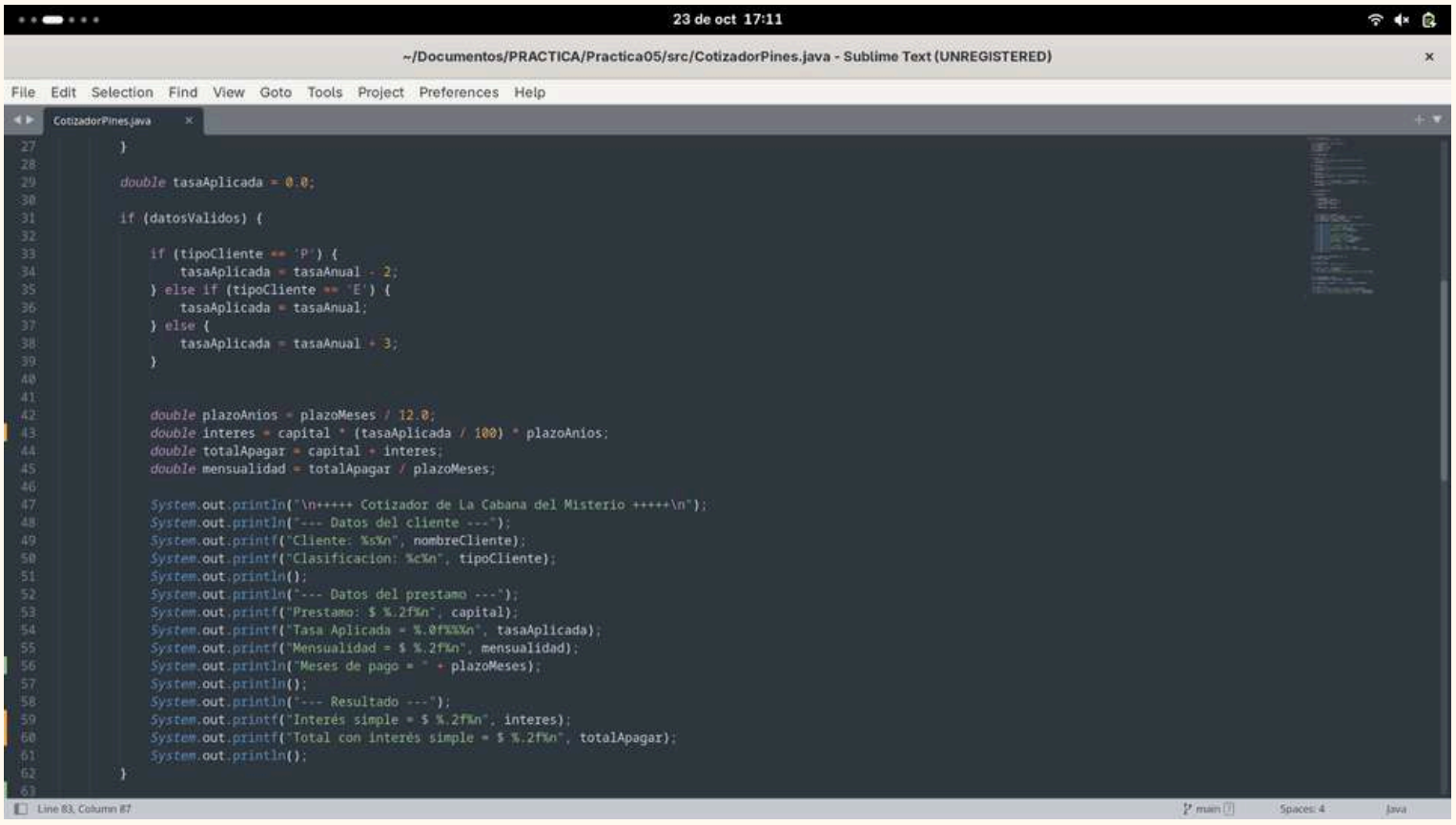
1. Tasa mensual: tasaMensual = tasaAplicada / 100 / 12. Primero convierto el porcentaje a fracción (÷100) y luego lo reparto entre los 12 meses (÷12).
2. Saldo inicial: saldo = capital, porque antes del primer mes la deuda es igual al préstamo.
3. Ciclo for: usé un ciclo que da una vuelta por cada mes del plazo (mes va de 1 a plazoMeses). Elegí for y no while porque ya sé de antemano cuántas veces debe repetirse.
4. Dentro del ciclo: actualizo el saldo con saldo = saldo * (1 + tasaMensual) e imprimo en la misma vuelta el renglón con el número de mes y el saldo acumulado.
5. Al terminar: el saldo final es el total con interés compuesto, y el interés compuesto es totalCompuesto - capital.

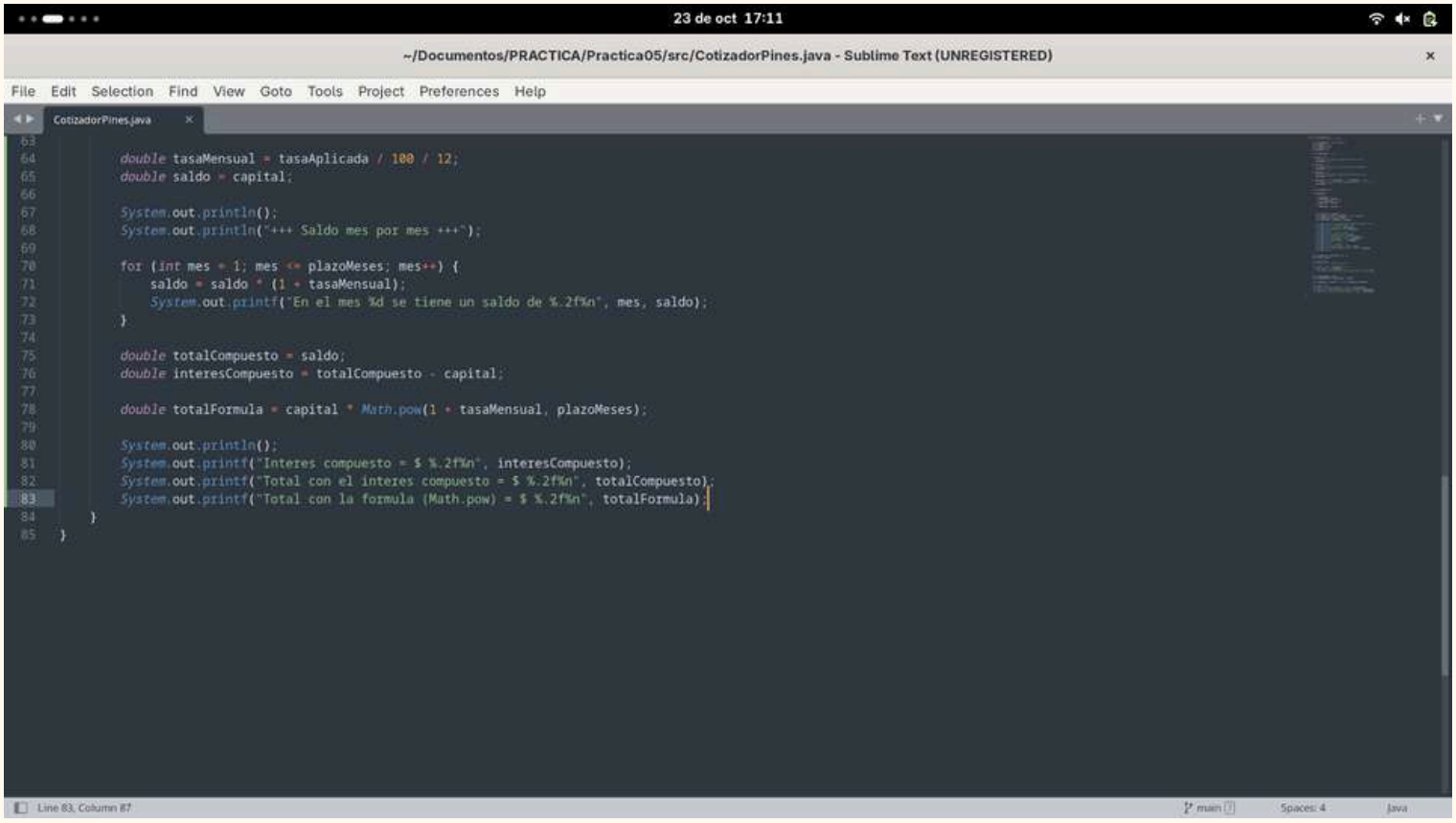
Como en este modelo el cliente no paga nada durante el préstamo, el saldo solo crece y no se resta ninguna mensualidad. Usé la misma variable saldo en cada vuelta para ir acumulando el resultado sin necesitar una variable por mes.

Después ya realice la parte dos, donde multiplicamos el saldo por (1 + tasaMensual) una vez por mes equivale a elevar ese factor a la potencia del número de meses. Por eso implementé también la fórmula directa que aparece en mi código, ahorita agrego mi código para que nos demos cuenta de estas cosas y no solo queden como palabras nada más, pero primero terminando con la explicación. Imprimí ambos resultados, el del ciclo y el de la fórmula, para compararlos. Ambos coinciden, lo cual confirma que el ciclo está bien implementado. La diferencia es que el ciclo me permite ver el saldo mes por mes, mientras que Math.pow calcula el resultado final de una sola vez.

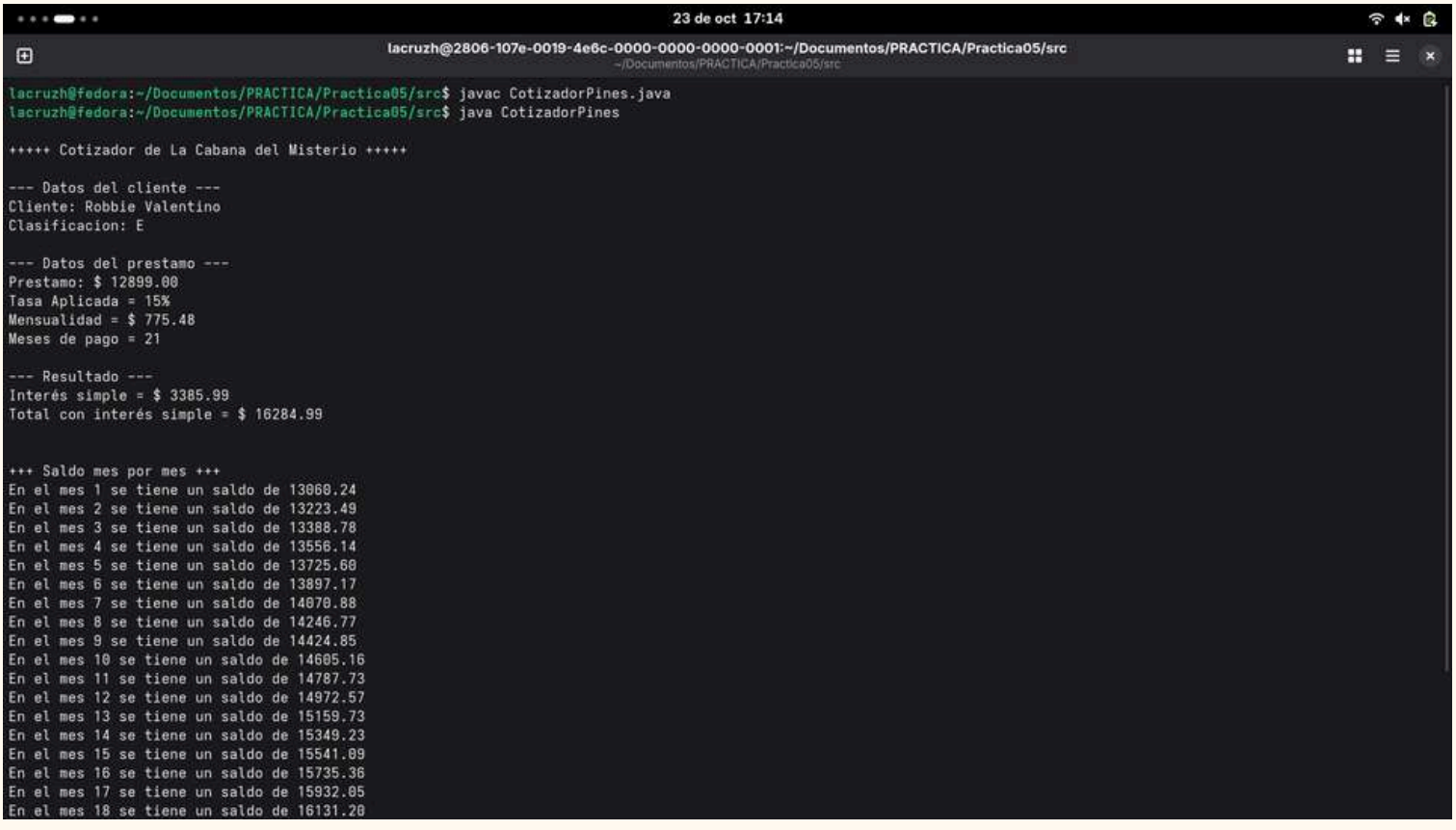
Puede ser repetitivo, pero bueno volví a agregar las fotos de mi código para que se vea bien cada uno de los puntos que abarque y ademas checar bien los datos que agregue para poder cumplir con lo requerido, una disculpa si te hice leer dos veces por el hecho de poner dos veces mi código ;(

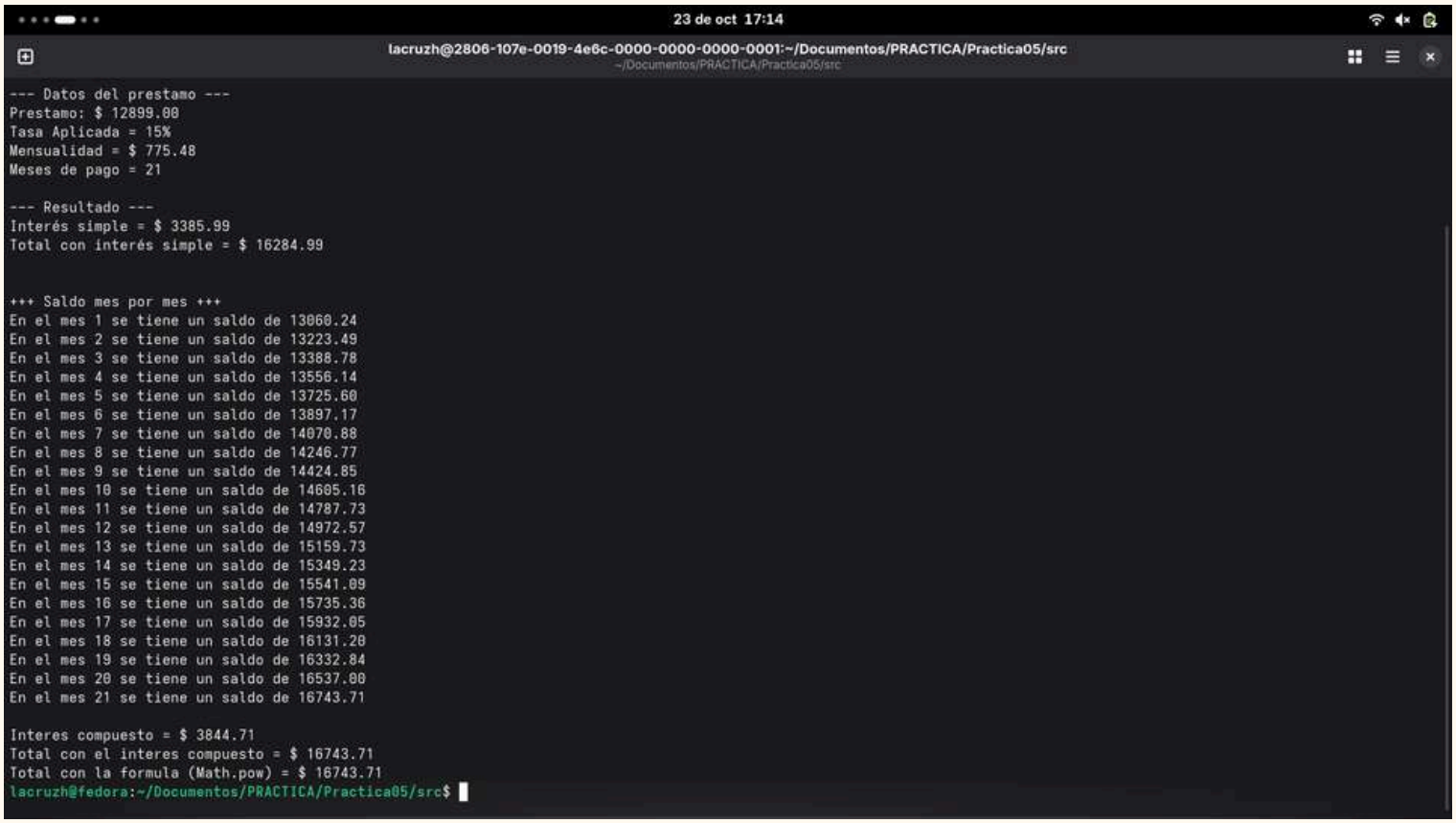

23 de oct 17:10
~/Documentos/PRACTICA/Practica05/src/CotizadorPines.java - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
CotizadorPines.java
public class CotizadorPines {
public static void main(String [] args) {
String nombreCliente = "Robbie Valentino";
char tipoCliente = 'E';
double capital = 12899.0;
double tasaAnual = 15.0;
int plazoMeses = 21;
boolean datosValidos = true;
if (capital <= 0) {
System.out.println("Error: el capital debe ser mayor que 0");
datosValidos = false;
}
if (tasaAnual < 0) {
System.out.println("Error: la tasa anual no puede ser negativa");
datosValidos = false;
}
if (plazoMeses <= 0) {
System.out.println("Error: el plazo debe ser de al menos 1 mes");
datosValidos = false;
}
if (tipoCliente != 'P' && tipoCliente != 'E' && tipoCliente != 'R') {
System.out.println("Error: el tipo de cliente debe ser 'P', 'E' o 'R'.");
datosValidos = false;
}
double tasaAplicada = 0.0;
if (datosValidos) {
if (tipoCliente == 'P') {
tasaAplicada = tasaAnual - 2;
} else if (tipoCliente == 'E') {
tasaAplicada = tasaAnual;
} else {
Line 83, Column 87
main
Spaces: 4
Java


23 de oct 17:11
~/Documentos/PRACTICA/Practica05/src/CotizadorPines.java - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
CotizadorPines.java
}
double tasaAplicada = 0.0;
if (datosValidos) {
if (tipoCliente == 'P') {
tasaAplicada = tasaAnual - 2;
} else if (tipoCliente == 'E') {
tasaAplicada = tasaAnual;
} else {
tasaAplicada = tasaAnual + 3;
}
double plazoAnios = plazoMeses / 12.0;
double interes = capital * (tasaAplicada / 100) * plazoAnios;
double totalApagar = capital + interes;
double mensualidad = totalApagar / plazoMeses;
System.out.println("\n+++++ Cotizador de La Cabana del Misterio +++++\n");
System.out.println("--- Datos del cliente ---");
System.out.printf("Cliente: %s%n", nombreCliente);
System.out.printf("Clasificacion: %c%n", tipoCliente);
System.out.println();
System.out.println("--- Datos del prestamo ---");
System.out.printf("Prestamo: $ %.2f%n", capital);
System.out.printf("Tasa Aplicada = %.0f%%%n", tasaAplicada);
System.out.printf("Mensualidad = $ %.2f%n", mensualidad);
System.out.println("Meses de pago = " + plazoMeses);
System.out.println();
System.out.println("--- Resultado ---");
System.out.printf("Interés simple = $ %.2f%n", interes);
System.out.printf("Total con interés simple = $ %.2f%n", totalApagar);
System.out.println();
}
Line 83, Column 87
main
Spaces: 4
Java



Observamos que los datos que aparecen en el código estan bien y no hay problema con los datos de Robbie, junto con lo que vamos a ver a continuación que sale en la terminal comprobamos que el códgio como lo que suelta la terminal es correcto.





Todo salió bien y los datos coinciden con los del recuadro de la actividad. También se nota que el interés compuesto ($3,844.71) es mayor que el simple ($3,385.99), lo cual es lo esperado: en el compuesto los intereses también generan intereses.

Bueno al final pues todo salio bien, la verdad esta parte tiene un pequeño guiño, que bueno lo explicare más adelante en la conclusión, por ahora, terminamos la actividad dos de dos, y bueno la parte 2 esta hecha, en esta la verdad tuve una que otra complicación ya que hay algunos datos que no eran familiares, o que no entendía bien cada una de las cosas, pero por ahora me quedo con que esta resuelta que es lo primordial, para esto yo creo que tengo que repasar cada una de las formas que se ocuparon en mi código y poder conocerlas mejor y que no haya tantos problemas después, porque bien la resolví y todo pero si no me queda claro cada una de las cosas, después al ver las cosas en el examen donde es más importante conocer cada cosa y poder realizar bien el examen, en este caso creo que tomare en cuenta el apunto que nos dio el ayudante en el laboratorio como referencia para repasar más y además también leer las notas que nos subió el profesor para repasar cada una de las cosa que me quedaron duda en esta práctica, pero bueno, ahora si para no alargarme más pasemos a lo último, la bonita conclusión.

CONCLUSIÓN

Bueno llegamos pareciera que no pero logré llegar al final del reporte, con esto en mano, pues ahora si explico todo lo que paso en esta, primero antes que nada, retomamos el inicio donde no entendí bien as cosas, con una que otra explicación más pues pude lograr entender maso menos las cosas, pero siguiendo con un poco de dudas, bueno el caso fue que al final llegamos a completar cada una de las actividades correspondientes, en este caso fue mucha en el labo, la verdad que si no fuera por el ayudante de laboratorio, esta práctica no hubiera logrado avanzar mucho, porque la verdad me perdí en muchas partes, pero justamente con la ayuda que recibí, pude realizar cada una de las actividades, creo que en muchas partes del reporte repito varias veces lo mismo, creo que eso ya es más problema mio con la redacción, prometo que en la siguiente práctica ahora si mejorare mi redacción, eh intentado pero las demás materias no me dan tiempo ni de respirar, pero bueno eso ya es tema aparte, pero si prometo mejorar la redacción paar que se pueda entender mejores las cosas y que no se vea que repito mucho cada una de las cosas. Siguiendo con la parte 2 pues en este fue el caso particular de dos cosas que no comente antes porque justamente queria que se viera bien aqui todo y no opacar con mucha información los demás puntos y dejarlos bien definidos cada una de las actividades, bueno una de las primeras esque si nos fijamos atentamente vemos que en las capturas que hice de mi códgio en la parte del cotizador, en una captura se ve distinto el nombre de las variables, esto cambia porque al principio yo tenia declarada con otro nombre pero al ir realizando la ultima parte de la practica vi que se tenian que guardar con otros nombres asi que ese fue el cambio que realice, y porque unas cosas no aparecen antes y despues si, más que nada fue porque al final pues lei lo útlimo que tenia que poner y que tenia que soltar la terminal, ya con eso eso pues eso fue una cosa, la siguiente fue que si nos fijamos atentamente en el código hice uso de muchas cosas de parte de la ayuda del código que me eseñaron en el labo, la verdad si soy sincero estaba perdido en el momento que realice es parte asi que por eso agregue o bueno mas que agregue copie esas parte porque me sentia perdido, lo cual lo que hice fue que justamente con cada parte entenderla para que no solo fuera que estuviera copiando si no que igualmente este entendiendo por que el uso de esta, con eso aclarado creo que la práctica estuvo muy buena la verdad, el pequeño problema fue que no habia entendido bien el funcionamiento de cada cosa, pero ahora que maso menos puede decir que conozco las cosas tendria que practicar más para las siguientes que viene pero además no dejarlo porque si este tema se me va las consecuencias seran muy grabes asi que fue todo por esta práctica, igualmente agradezco la dedicacion que tiene cada uno para podernos apoyar, y mas que nada por el apoyo brindado con todos mis problemas y que igualmente puede que sean muy novatos, pero agradezco la paciencia para poderme guiarme y poder entregar y entender las cosas. Nos vemos en la próxima práctica ;).

